

Lab 1: Setting Up a Virtual Machine Using Oracle VirtualBox

Objective:

To create and configure an Ubuntu-based virtual machine (VM) on a local system using Oracle VirtualBox.

Tools Used:

- Oracle VirtualBox v7.x
- Ubuntu 22.04 ISO (64-bit)

Implementation:

Oracle VirtualBox was downloaded from the official website (version 7.0.20 for Windows) and installed on the host machine (Windows). A new virtual machine named ubuntu was created with the following specifications:

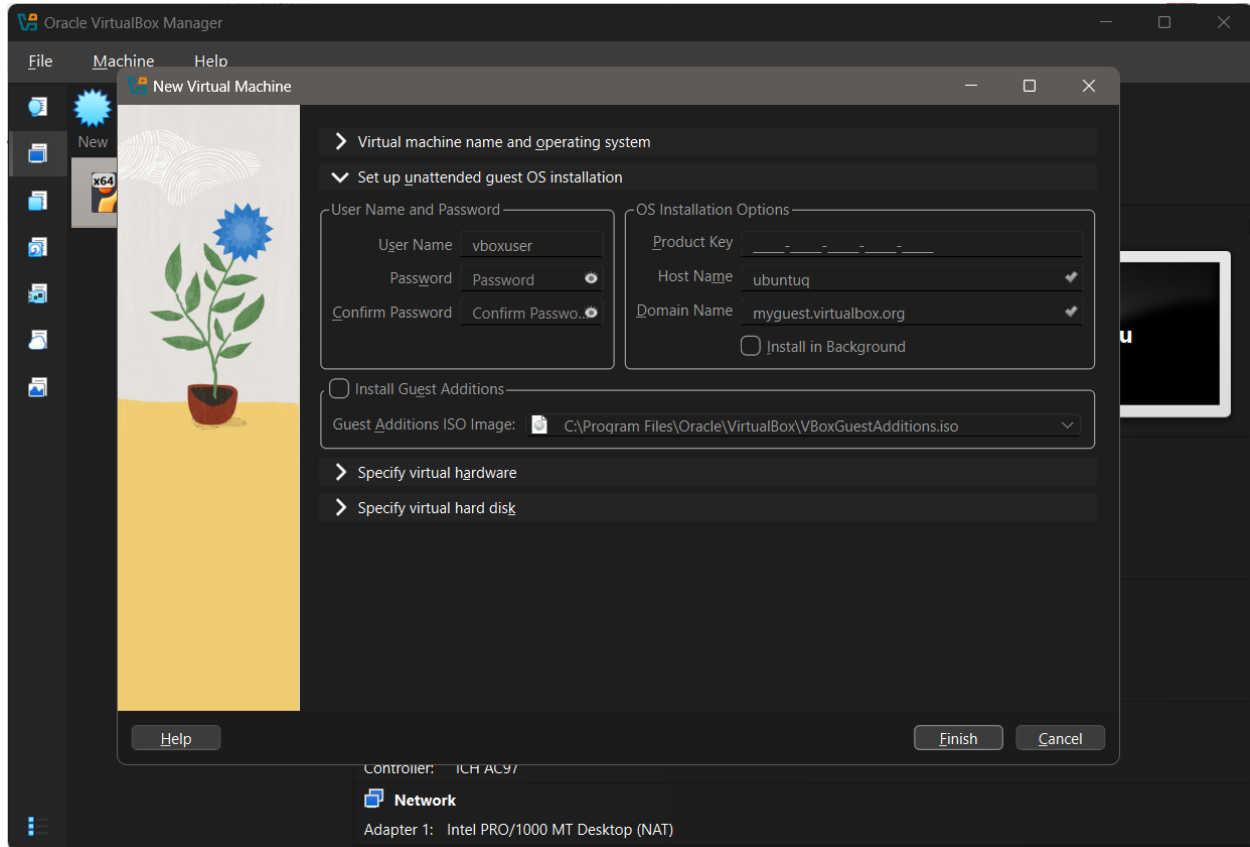
- **Guest OS:** Ubuntu (64-bit)
- **Base Memory:** Initially set to 2048 MB, later increased to 4594 MB
- **Processors:** 1 CPU
- **Graphics Controller:** VMSVGA with 128 MB video memory
- **Storage:** 45.53 GB dynamically allocated VDI disk (ubuntu.vdi)
- **Network:** Intel PRO/1000 MT Desktop (NAT)
- **Guest Additions:** Automatically installed using VBoxGuestAdditions.iso

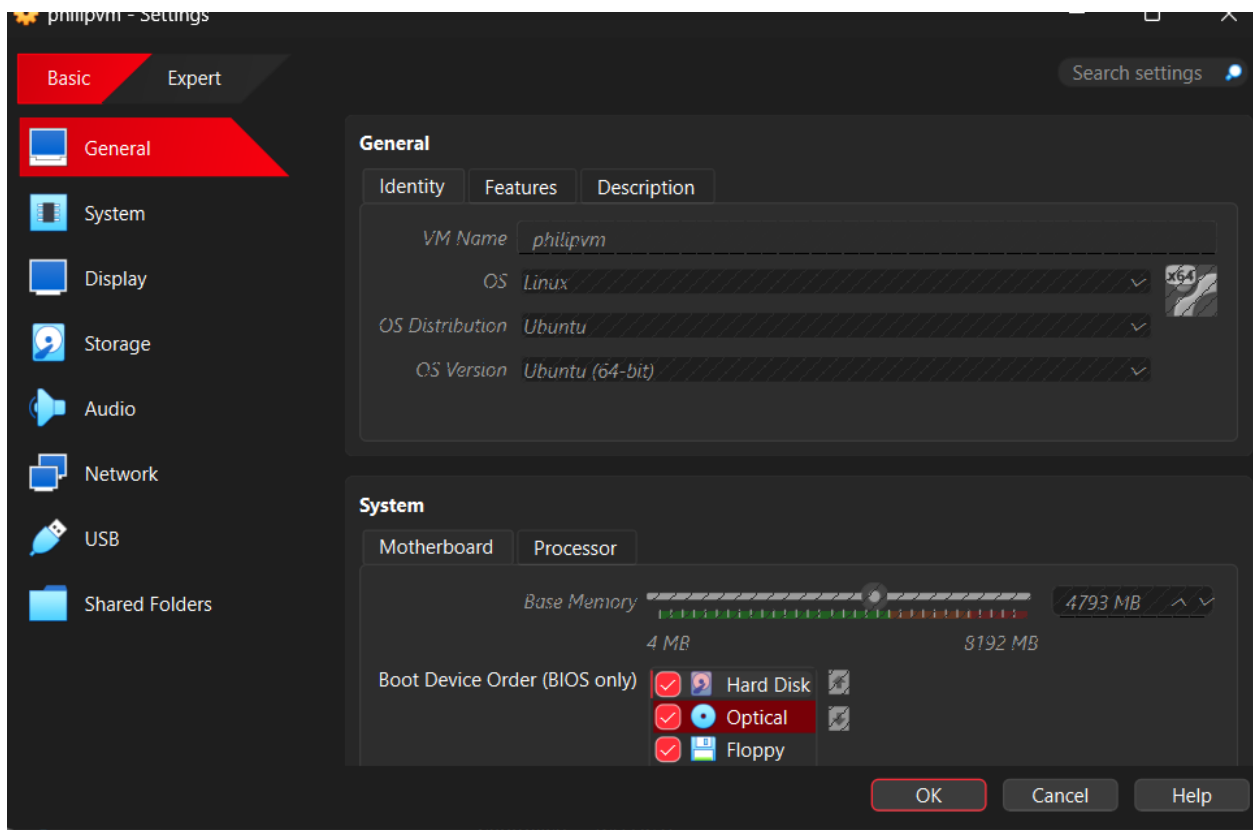
An unattended installation was configured using:

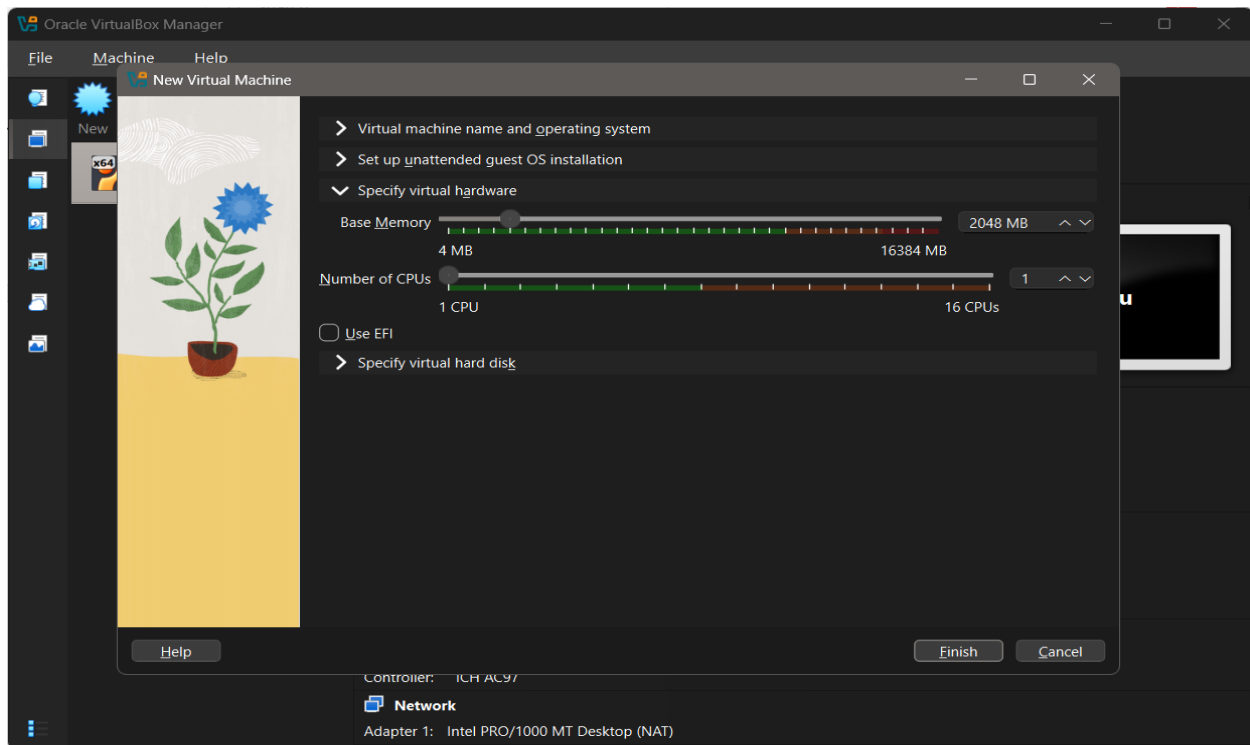
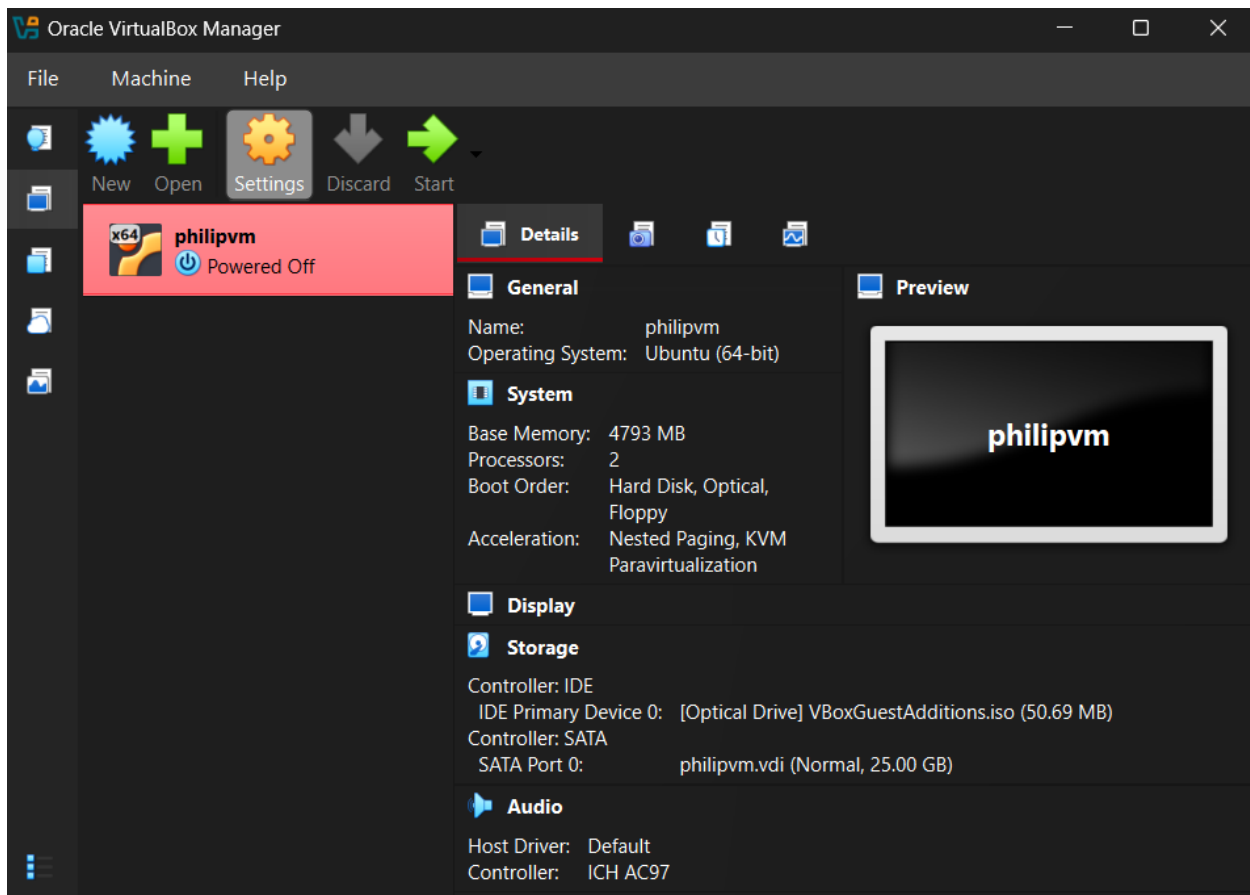
- **Username:** philip
- **Password:** Password
- **Hostname:** ubuntu
- **Domain:** myguest.virtualbox.org

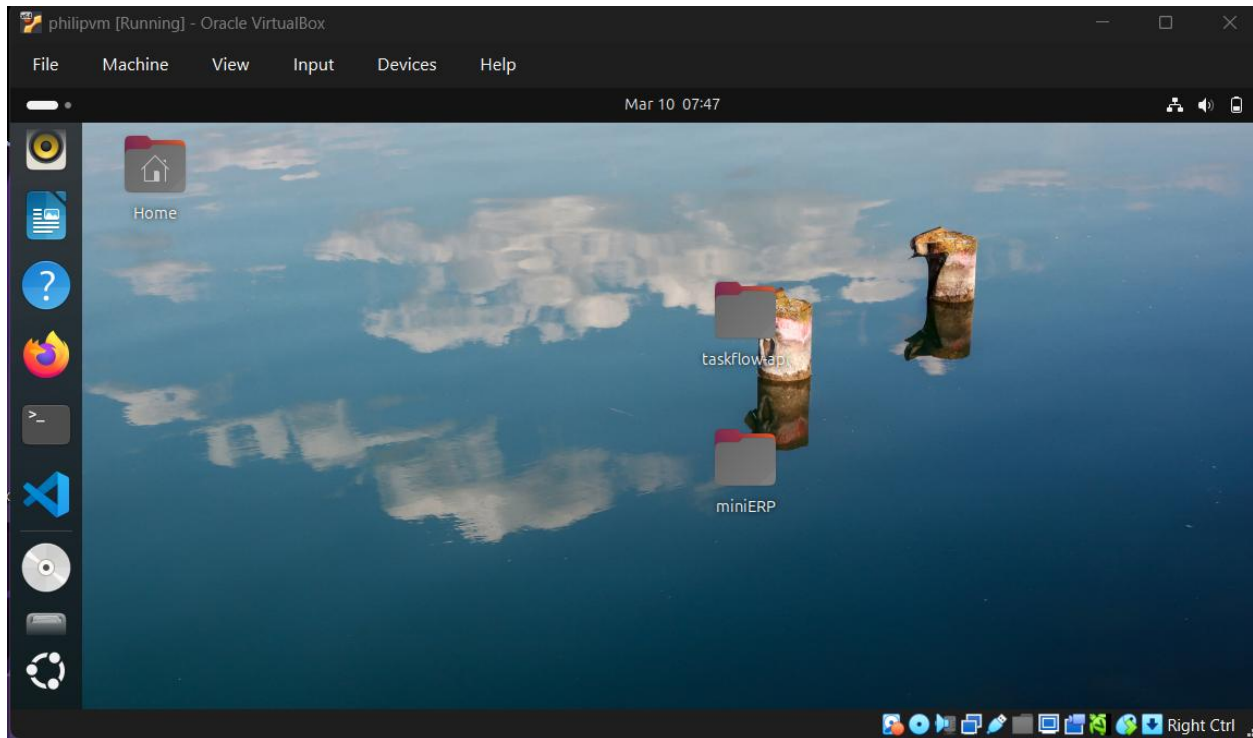
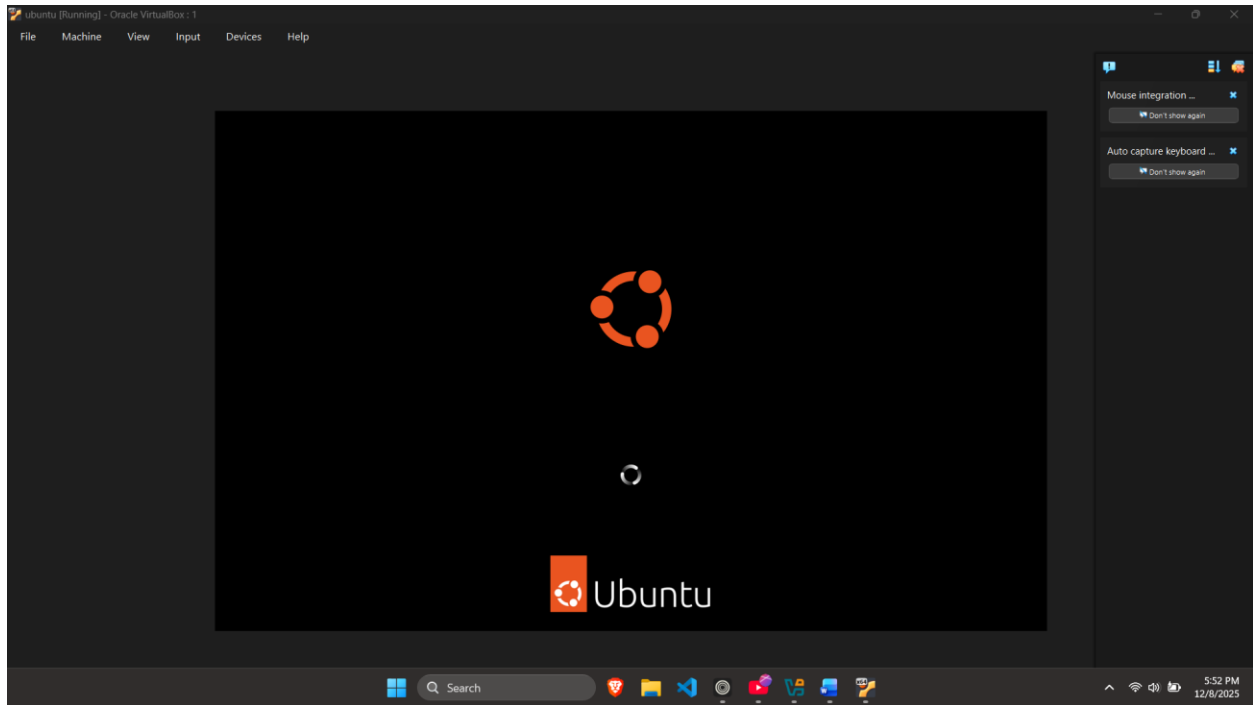
Conclusion:

A local Ubuntu virtual machine named ubuntu was successfully created and configured using Oracle VirtualBox. The VM booted correctly into Ubuntu and operated as a functional isolated Linux environment, confirming the successful implementation of local virtualization









Lab 2: Deploying and configuring an EC2 Instance on AWS

Amazon EC2 (Elastic Compute Cloud) provides scalable computing capacity in the AWS cloud.

Objective:

To launch an Amazon EC2 instance and deploy a simple Node.js web server.

Tools Used:

- AWS EC2 (Amazon Linux 2023 AMI)
- Node.js via NVM

Implementation:

The following steps were carried out to deploy and configure the EC2 instance:

1. Launched an EC2 Instance
 - Chose t2.micro instance type.
 - Used an existing key pair: **philip-key.pem** for SSH authentication.
2. Established SSH Connection
 - Opened Command Prompt on the local Windows machine (C:\Users\ASUS\Downloads).
 - Ran the SSH command:

```
ssh -i "philip-key.pem" ec2-user@ec2-13-203-154-0.ap-south1.compute.amazonaws.com
```
3. Created a new directory named testphilip and navigated into it
4. Downloaded and installed NVM (Node Version Manager)
5. Installed Node.js via NVM
6. initialized Node.js Project
 - Ran `npm init -y` to generate a default package.json file.
7. Created a Simple HTTP Server
 - Created index.js using the nano editor to create a simple server.
8. Configured Network Access
9. Verified Web Access

- Opened a browser and navigated to: <http://13.203.154.0:3000>
- Successfully saw the response: “Hello from Node Server”

Conclusion:

Amazon EC2 instance running Amazon Linux 2023 was successfully launched in the ap-south-1 region. The server listened on port 3000 and responded with “Hello from Node Server”.

```

ec2-user@ip-172-31-15-74:~/t  ×  +  ▾
C:\Users\phili\Downloads>ssh -i "philip-key.pem" ec2-user@ec2-65-2-57-61.ap-south-1.compute.amazonaws.com
The authenticity of host 'ec2-65-2-57-61.ap-south-1.compute.amazonaws.com (65.2.57.61)' can't be established.
ED25519 key fingerprint is SHA256:PPgoiJNs8VMQGGuUQslGQg3Ppctw0jPa7mRfAxwSKD7k.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'ec2-65-2-57-61.ap-south-1.compute.amazonaws.com' (ED25519) to the list of known hosts.

      #_
     _/ \_ #####_   Amazon Linux 2023
    _/  \_ #####\
   _/    \_###|
  _/      \#/___|_  https://aws.amazon.com/linux/amazon-linux-2023
 _/        V_! '--->
NNN      _/ \_ /
 _/  \_ / \_ /
 _/m/! '--->

[ec2-user@ip-172-31-15-74 ~]$
[ec2-user@ip-172-31-15-74 ~]$ mkdir newproject
[ec2-user@ip-172-31-15-74 ~]$ cd newproject
[ec2-user@ip-172-31-15-74 newproject]$ curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.1/install.sh | bash
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100 16563  100 16563    0     0  66423      0  --:--:-- --:--:-- --:--:--  66786
=> Downloading nvm as script to '/home/ec2-user/.nvm'

=> Appending nvm source string to /home/ec2-user/.bashrc
=> Appending bash_completion source string to /home/ec2-user/.bashrc
=> Close and reopen your terminal to start using nvm or run the following to use it now:

export NVM_DIR="$HOME/.nvm"

% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
100 16563  100 16563    0     0  66423      0  --:--:-- --:--:-- --:--:--  66786
=> Downloading nvm as script to '/home/ec2-user/.nvm'

=> Appending nvm source string to /home/ec2-user/.bashrc
=> Appending bash_completion source string to /home/ec2-user/.bashrc
=> Close and reopen your terminal to start using nvm or run the following to use it now:

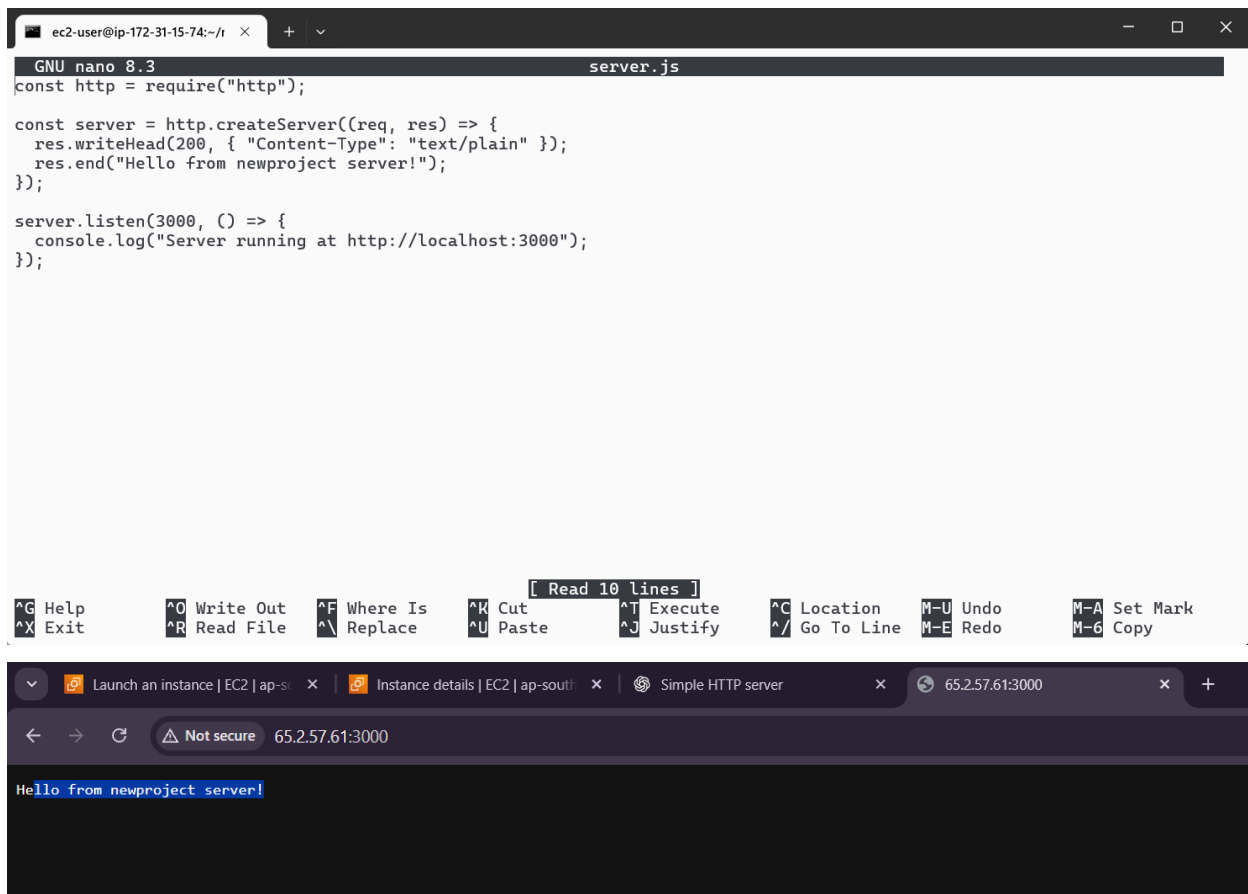
export NVM_DIR="$HOME/.nvm"
[ -s "$NVM_DIR/nvm.sh" ] && \. "$NVM_DIR/nvm.sh" # This loads nvm
[ -s "$NVM_DIR/bash_completion" ] && \. "$NVM_DIR/bash_completion" # This loads nvm bash_completion
[ec2-user@ip-172-31-15-74 newproject]$ source ~/.nvm/nvm.sh
source ~/.bashrc
[ec2-user@ip-172-31-15-74 newproject]$ nvm install --lts
Installing latest LTS version.
Downloading and installing node v24.11.1...
Downloading https://nodejs.org/dist/v24.11.1/node-v24.11.1-linux-x64.tar.xz...
##### 100.0%
Computing checksum with sha256sum
Checksums matched!
Now using node v24.11.1 (npm v11.6.2)
Creating default alias: default -> lts/* (-> v24.11.1)
[ec2-user@ip-172-31-15-74 newproject]$ node -v
v24.11.1
npm -v
11.6.2

```

```
[ec2-user@ip-172-31-15-74 newproject]$ npm init -y
Wrote to /home/ec2-user/newproject/package.json:
```

```
{
  "name": "newproject",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

```
[ec2-user@ip-172-31-15-74 newproject]$ nano server.js
[ec2-user@ip-172-31-15-74 newproject]$ node server.js
Server running at http://localhost:3000
```



Lab 3: Creating and managing an S3 Bucket on AWS

Amazon S3 (Simple Storage Service) is used for object storage in AWS.

Objective:

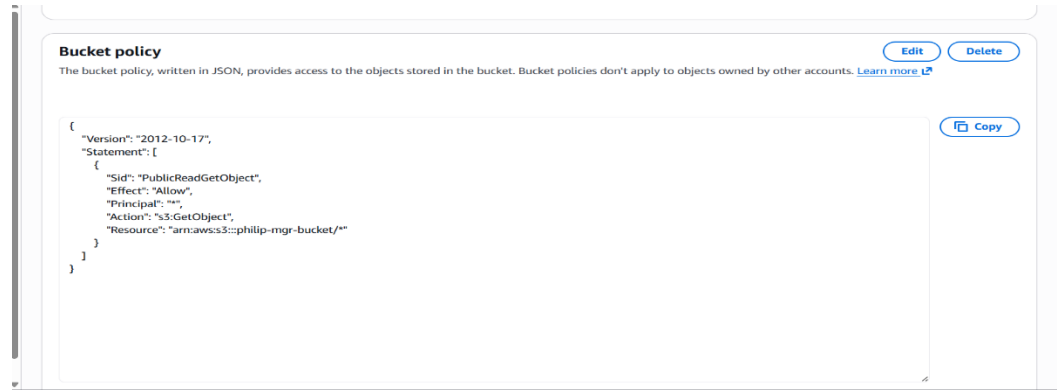
To create an S3 bucket, upload static website files, configure public access, and serve a live webpage.

Tools Used:

- AWS S3 Console
- Static website files (HTML, CSS, JS, images)

Implementation:

1. Created an S3 Bucket
 - Go to AWS S3 Console → Create bucket
 - Bucket name: Philip-mgr.-bucket
 - Block Public Access: Uncheck (to allow public read)
2. Uploaded Website Files
 - Clicked on the bucket → Upload
 - Add folder: example
 - Uploaded templatemo_603_nexaverse folder containing:
 - index.html
 - tooplate-nexaverse.css
 - tooplate-living-scripts.js
 - /images/ directory with 10+ JPEG files
3. Configure Bucket Policy for Public Read
 - Go to Permissions → Bucket Policy
 - Added the Permissions Policy



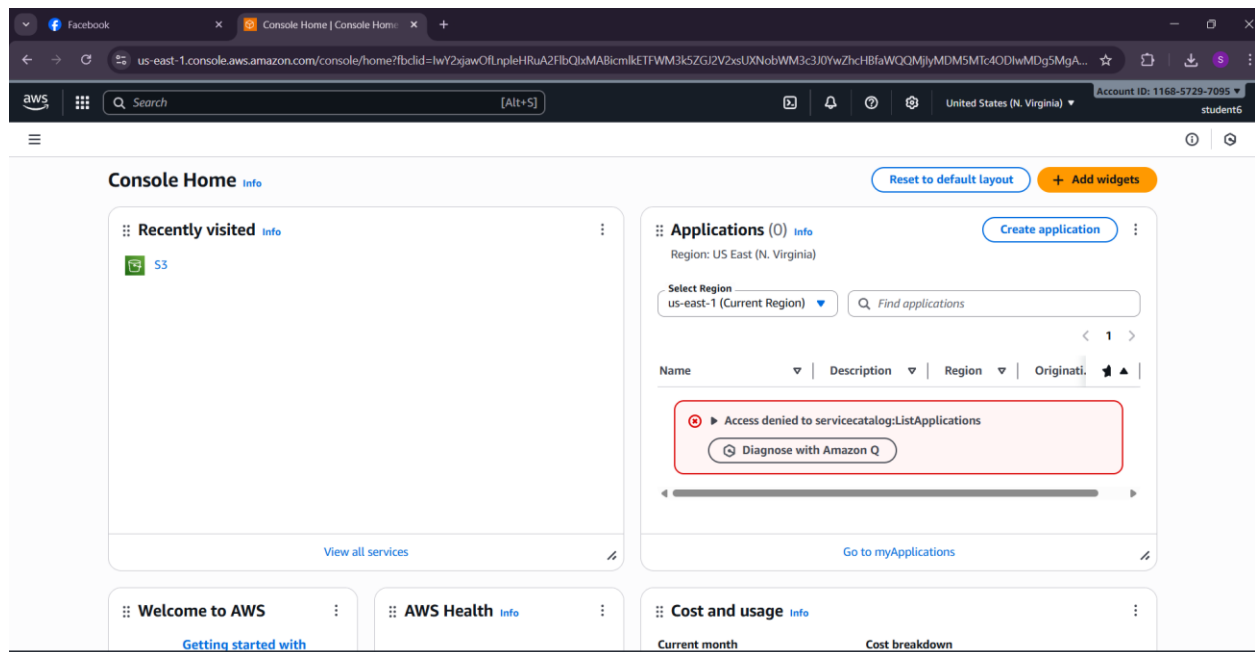
4. Accessed the Live Website

- Object URL for index.html:

Opened in browser → Displays “NEXA VERSE” website with images and styling

Conclusion:

A public S3 bucket named example was created and static website folder was uploaded. The static website was successfully accessed via the public URL, confirming that S3 can host and serve static web content globally.



philip-mgr-bucket

philip-mgr-bucket

Info

Objects

Metadata

Properties

Permissions

Metrics

Management

Access Points

Objects (2)

Copy S3 URI

Copy URL

Download

Open

Delete

Actions

Create folder

Upload

Find objects by prefix

< 1 >

	Name	Type	Last modified	Size	Storage class
☐	example/	Folder	-	-	-
☐	mountains_front.png	png	December 5, 2025, 08:35:23 (UTC+05:45)	525.0 KB	Standard

S3 buckets | S3 | us-east-1

AWS S3 bucket policies

html template free download

+

us-east-1.console.aws.amazon.com/s3/buckets?region=us-east-1

Search

[Alt+S]

United States (N. Virginia)

Account ID: 1168-5729-7095

student6

Amazon S3

Buckets

General purpose buckets

Directory buckets

General purpose buckets (5)

Copy ARN

Empty

Delete

Create bucket

Find buckets by name

< 1 >

	Name	AWS Region	Creation date
☐	philip-mgr-bucket	US East (N. Virginia) us-east-1	December 5, 2025, 08:31:04 (UTC+05:45)
☐	prashantshah-demo	US East (N. Virginia) us-east-1	December 5, 2025, 08:31:13 (UTC+05:45)
☐	sandip-bernhardt-demo	US East (N. Virginia) us-east-1	December 5, 2025, 08:30:59 (UTC+05:45)
☐	satish-bernhardt	US East (N. Virginia) us-east-1	December 5, 2025, 08:31:02 (UTC+05:45)
☐	sugam-demo	US East (N. Virginia) us-east-1	December 5, 2025, 08:31:15 (UTC+05:45)

Account snapshot

Updated daily

View dashboard

Storage Lens provides visibility into storage usage and activity trends.

External access summary - new

Updated daily

Info

External access findings help you identify bucket permissions that allow public access or access from other AWS accounts.

philip-mgr-bucketmountains_front.png

mountains_front.pngInfo

Copy S3 URIDownloadOpen ↗Object actions ▼

PropertiesPermissionsVersions

Object overview

Owner

4b9af3b3587a2e4734951c7d181722790900558b7576861199c754566bd7e5d7

AWS Region

US East (N. Virginia) us-east-1

Last modified

December 5, 2025, 08:35:23 (UTC+05:45)

Size

525.0 KB

Type

png

Key

mountains_front.png

S3 URI

s3://philip-mgr-bucket/mountains_front.png

Amazon Resource Name (ARN)

arn:aws:s3:::philip-mgr-bucket/mountains_front.png

Entity tag (Etag)

eb678ac716b3e2faa2c13b1fe19274fb

Object URL

https://philip-mgr-bucket.s3.us-east-1.amazonaws.com/mountains_front.png

philip-mgr-bucketmountains_front.png

mountains_front.pngInfo

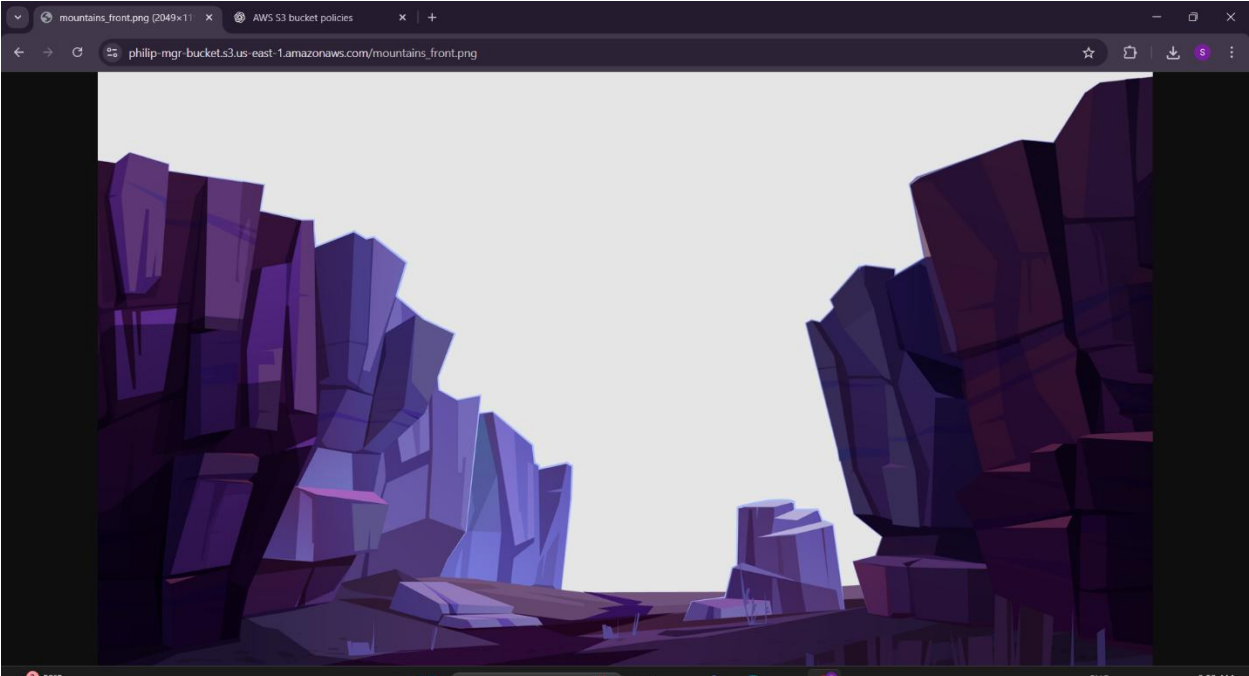
Copy S3 URIDownloadOpen ↗Object actions ▼

PropertiesPermissionsVersions

Access control list (ACL)Edit

Grant basic read/write permissions to AWS accounts. [Learn more](#) ↗

Grantee	Object	Object ACL
Object owner (your AWS account) Canonical ID: 4b9af3b3587a2e4734951c7d181722790900558b7576861199c754566bd7e5d7	Read	Read, Write
Everyone (public access) Group: http://acs.amazonaws.com/groups/global/AllUsers	⚠ Read	⚠ Read
Authenticated users group (anyone with an AWS account) Group: http://acs.amazonaws.com/groups/global/AuthenticatedUsers	-	-



philip-mgr-bucket

philip-mgr-bucket

Info

Objects

Metadata

Properties

Permissions

Metrics

Management

Access Points

Objects (2)

Copy S3 URI

Copy URL

Download

Open

Delete

Actions

Create folder

Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

< 1 >

☐	Name	Type	Last modified	Size	Storage class
☐	example/	Folder	-	-	-
☐	mountains_front.png	png	December 5, 2025, 08:35:23 (UTC+05:45)	525.0 KB	Standard

<

example/

📄 Copy S3 URI

Objects

Properties

Objects (16)

🔄 Copy S3 URI 📄 Copy URL ⬇ Download 🔗 Open 🗑 Delete ⌵ Actions 📁 Create folder 📶 Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

🔍 Find objects by prefix

< 1 > ⚙

☐	Name	Type	Last modified	Size	Storage class
☐	247-support.jpg	jpg	December 5, 2025, 08:48:52 (UTC+05:45)	21.6 KB	Standard
☐	about-our-team.jpg	jpg	December 5, 2025, 08:48:53 (UTC+05:45)	62.8 KB	Standard
☐	ai-mobile-app.jpg	jpg	December 5, 2025, 08:48:54 (UTC+05:45)	31.9 KB	Standard
☐	ai-optimized-templates.jpg	jpg	December 5, 2025, 08:48:55 (UTC+05:45)	24.8 KB	Standard
☐	brand-identity.jpg	jpg	December 5, 2025, 08:48:55 (UTC+05:45)	23.3 KB	Standard

<

Objects (16)

🔄 Copy S3 URI 📄 Copy URL ⬇ Download 🔗 Open 🗑 Delete ⌵ Actions 📁 Create folder 📶 Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

🔍 Find objects by prefix

< 1 > ⚙

☐	Name	Type	Last modified	Size	Storage class
☐	digital-strategy.jpg	jpg	December 5, 2025, 08:48:57 (UTC+05:45)	29.4 KB	Standard
☐	index.html	html	December 5, 2025, 08:49:04 (UTC+05:45)	36.6 KB	Standard
☐	managed-service.jpg	jpg	December 5, 2025, 08:48:58 (UTC+05:45)	17.9 KB	Standard
☐	motion-design.jpg	jpg	December 5, 2025, 08:48:59 (UTC+05:45)	26.1 KB	Standard
☐	process-optimized.jpg	jpg	December 5, 2025, 08:49:00 (UTC+05:45)	31.5 KB	Standard
☐	tech-audit.jpg	jpg	December 5, 2025, 08:49:01 (UTC+05:45)	34.3 KB	Standard
☐	templatemo-nexa-scripts.js	js	December 5, 2025, 08:49:03 (UTC+05:45)	8.1 KB	Standard
☐	templatemo-nexaverse.css	css	December 5, 2025, 08:49:02 (UTC+05:45)	32.7 KB	Standard

philip-mgr-bucket > example/ > index.html

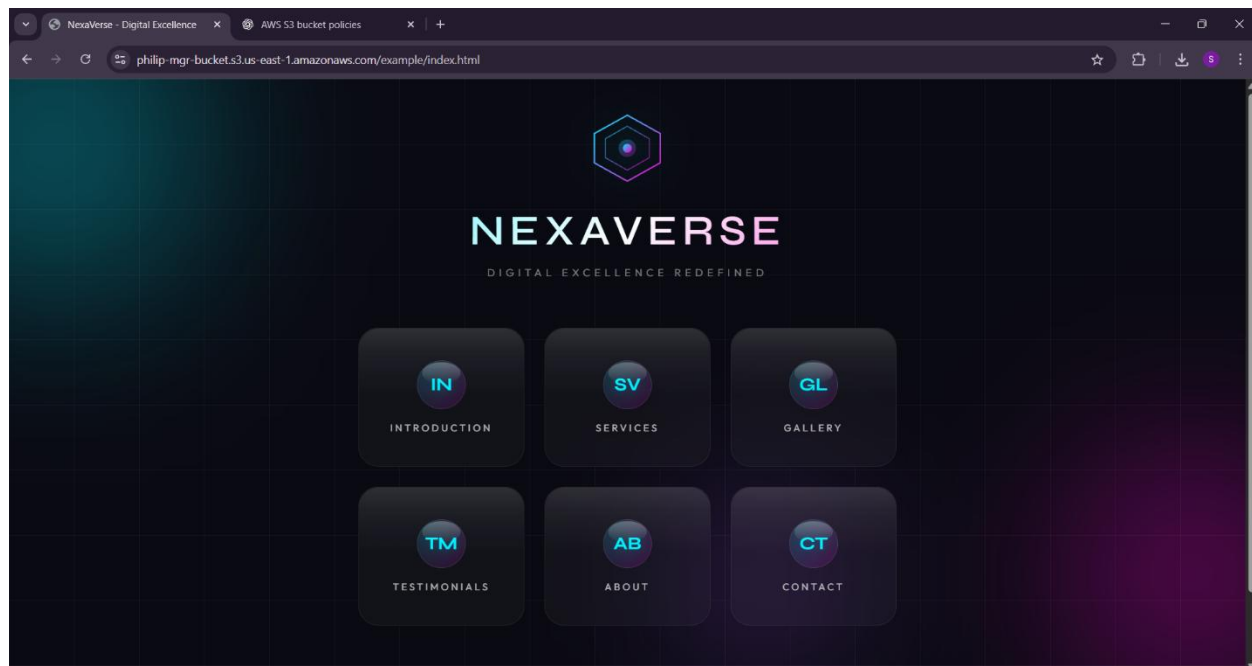
index.html [Info](#)

[Copy S3 URI](#) [Download](#) [Open](#) [Object actions](#)

[Properties](#) [Permissions](#) [Versions](#)

Object overview

Owner 4b9af3b3587a2e4734951c7d181722790900558b7576861199c754566bd7e5d7	S3 URI s3://philip-mgr-bucket/example/index.html
AWS Region US East (N. Virginia) us-east-1	Amazon Resource Name (ARN) arn:aws:s3:::philip-mgr-bucket/example/index.html
Last modified December 5, 2025, 08:49:04 (UTC+05:45)	Entity tag (Etag) 7d7761d60a2820a6360c622cce5ba71e
Size 36.6 KB	Object URL https://philip-mgr-bucket.s3.us-east-1.amazonaws.com/example/index.html
Type html	
Key example/index.html	



Lab 4: Integrating Amazon S3 with AWS Lambda and API Gateway

Objective:

To create an AWS Lambda function triggered via API Gateway that interacts with an Amazon S3 bucket.

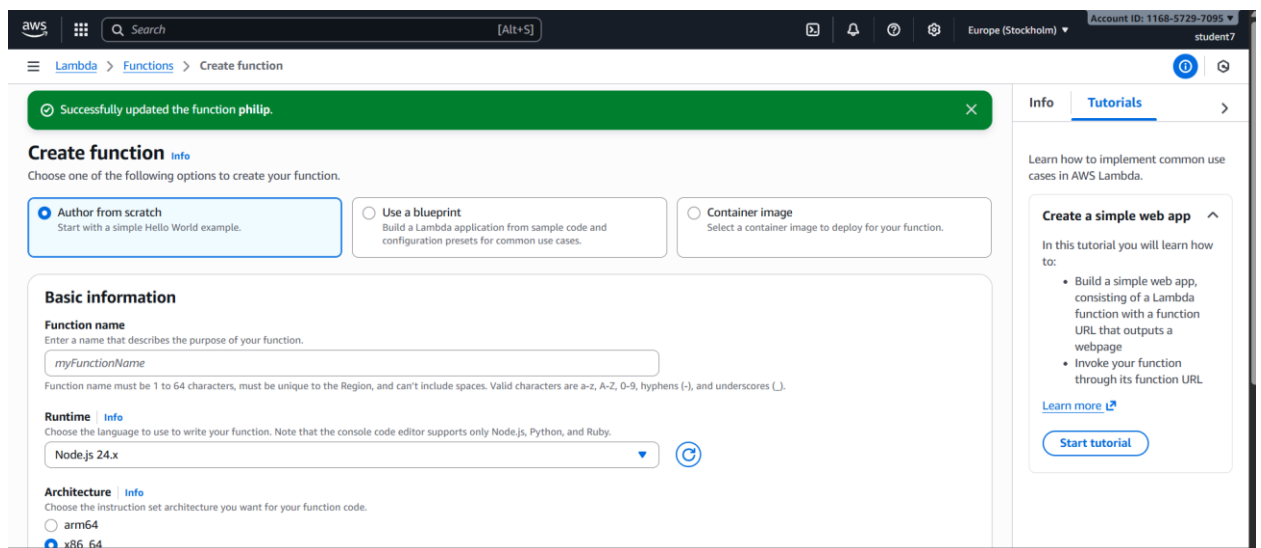
Tools Used:

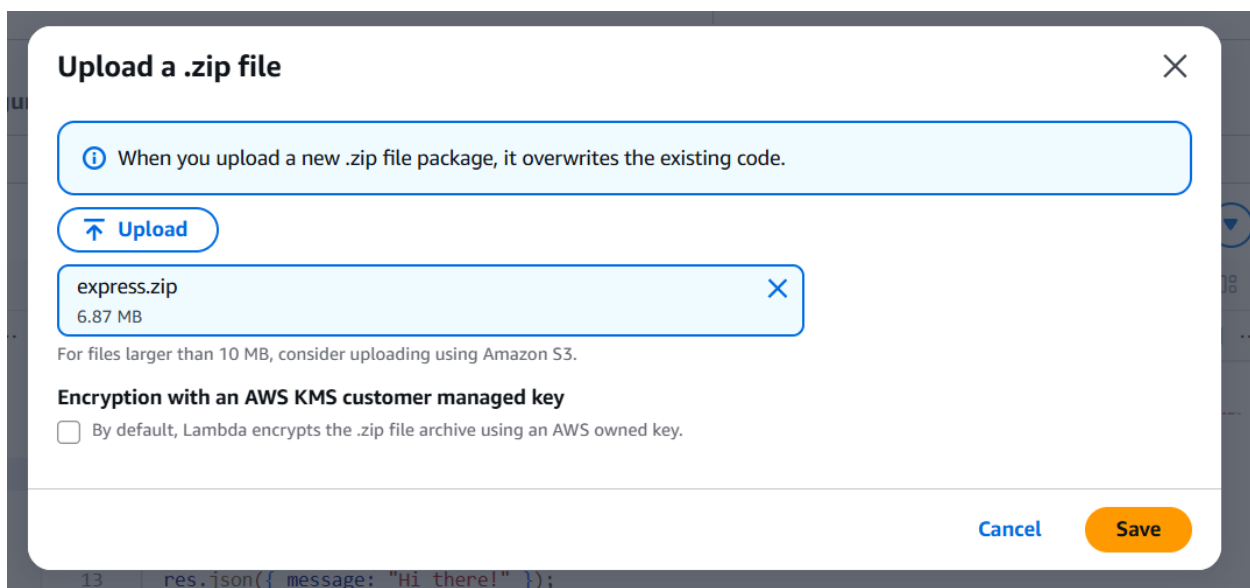
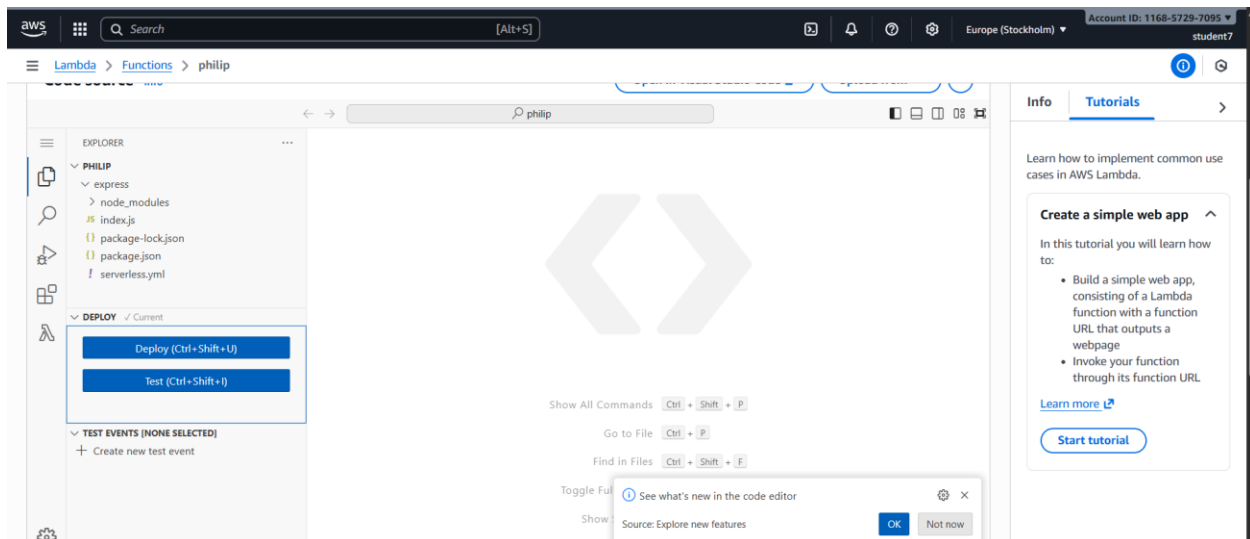
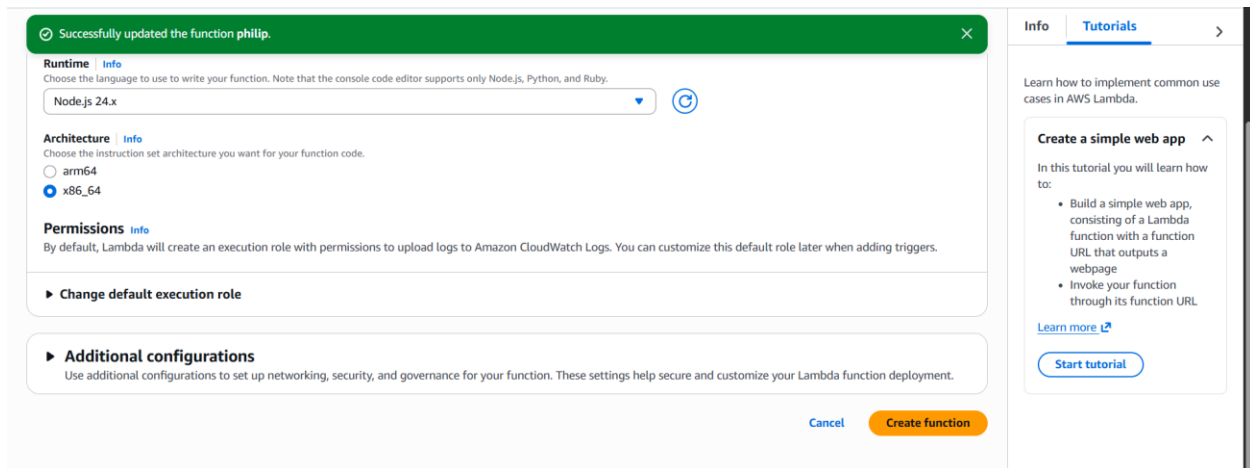
- AWS S3 (Simple Storage Service)
- AWS Lambda
- AWS API Gateway
- Node.js (for Lambda function)

Implementation:

1. Created a Lambda Function

- Runtime: Node.js 18.x
- Function name: Philip
- Assigned IAM role permissions for S3 read/write access.





2. Configured API Gateway

- Created a new HTTP API.
- Added a GET endpoint /get-file.
- Integrated the endpoint with the Lambda function.

The screenshot shows the AWS Management Console interface for creating a new HTTP API. The breadcrumb navigation at the top indicates the path: API Gateway > APIs > Create API > Create HTTP API. The left-hand navigation pane shows a progress bar with four steps: Step 1: Configure API, Step 2 - optional: Configure routes, Step 3 - optional: Define stages (which is the current step), and Step 4: Review and create. The main content area is titled 'Define stages - optional' and contains a 'Configure stages' section. This section explains that stages are independently configurable environments for API deployment. It features a 'Stage name' input field with the value '\$default', an 'Auto-deploy' toggle switch that is turned on, and a 'Remove' button. Below the input field is an 'Add stage' button. At the bottom right of the main content area are 'Cancel', 'Previous', and 'Next' buttons.

Step 1: Configure API
Step 2 - optional: Configure routes
Step 3 - optional: Define stages
Step 4: Review and create

Define stages - optional

Configure stages [info](#)

Stages are independently configurable environments that your API can be deployed to. You must deploy to a stage for API configuration changes to take effect, unless that stage is configured to autodeploy. By default, all HTTP APIs created through the console have a default stage named \$default. All changes that you make to your API are autodeployed to that stage. You can add stages that represent environments such as development or production.

Stage name: Auto-deploy: ☒ Remove

Add stage

Cancel Previous Next

The screenshot shows the AWS Management Console interface for creating a new HTTP API, specifically the 'Configure API' step. The breadcrumb navigation at the top indicates the path: API Gateway > APIs > Create API > Create HTTP API. The left-hand navigation pane shows a progress bar with four steps: Step 1: Configure API (which is the current step), Step 2 - optional: Configure routes, Step 3 - optional: Define stages, and Step 4: Review and create. The main content area is titled 'Configure API' and contains an 'API details' section. This section includes an 'API name' input field with the value 'philipapi'. Below this is the 'IP address type' section, which has two options: 'IPv4' (selected) and 'Dualstack'. The 'Integrations' section shows '(0)' integrations and an 'Add integration' button. At the bottom right of the main content area are 'Cancel', 'Review and create', and 'Next' buttons.

Step 1: Configure API
Step 2 - optional: Configure routes
Step 3 - optional: Define stages
Step 4: Review and create

Configure API

API details

API name
An HTTP API must have a name. The name is a non-unique value you use to identify and organize your APIs. To programmatically refer to this API, use the API ID that API Gateway generates for you.

IP address type [Info](#)
Select the type of IP addresses that can invoke the default endpoint for your API. You don't need to redeploy your API for the update to take effect.

☒ IPv4
Includes only IPv4 addresses.

☐ Dualstack
Includes IPv4 and IPv6 addresses.

Integrations (0) [Info](#)
Specify the backend services that your API will communicate with. These are called integrations. For a Lambda integration, API Gateway invokes the Lambda function and responds with the response from the function. For an HTTP integration, API Gateway sends the request to the URL that you specify and returns the response from the URL.

Add integration

Cancel Review and create Next

- Step 1
Configure API
- Step 2 - optional
Configure routes
- Step 3 - optional
Define stages
- Step 4
Review and create

Configure routes - optional

Configure routes [Info](#)

API Gateway uses routes to expose integrations to consumers of your API. Routes for HTTP APIs consist of two parts: an HTTP method and a resource path (e.g., GET /pets). You can define specific HTTP methods for your integration (GET, POST, PUT, PATCH, HEAD, OPTIONS, and DELETE) or use the ANY method to match all methods that you haven't defined on a given resource.

[Add route](#)

[Cancel](#) [Review and create](#) [Previous](#) [Next](#)

- Step 3 - optional
Define stages
- Step 4
Review and create

API name

An HTTP API must have a name. The name is a non-unique value you use to identify and organize your APIs. To programmatically refer to this API, use the API ID that API Gateway generates for you.

philipapi

IP address type [Info](#)

Select the type of IP addresses that can invoke the default endpoint for your API. You don't need to redeploy your API for the update to take effect.

☒ IPv4

Includes only IPv4 addresses.

☐ Dualstack

Includes IPv4 and IPv6 addresses.

Integrations (1) [Info](#)

Specify the backend services that your API will communicate with. These are called integrations. For a Lambda integration, API Gateway invokes the Lambda function and responds with the response from the function. For an HTTP integration, API Gateway sends the request to the URL that you specify and returns the response from the URL.

Lambda

AWS Region

eu-north-1

Lambda function

Version

2.0

[Learn more](#)

[Remove](#)

[Add integration](#)

[Cancel](#) [Review and create](#) [Next](#)

EXPLORER

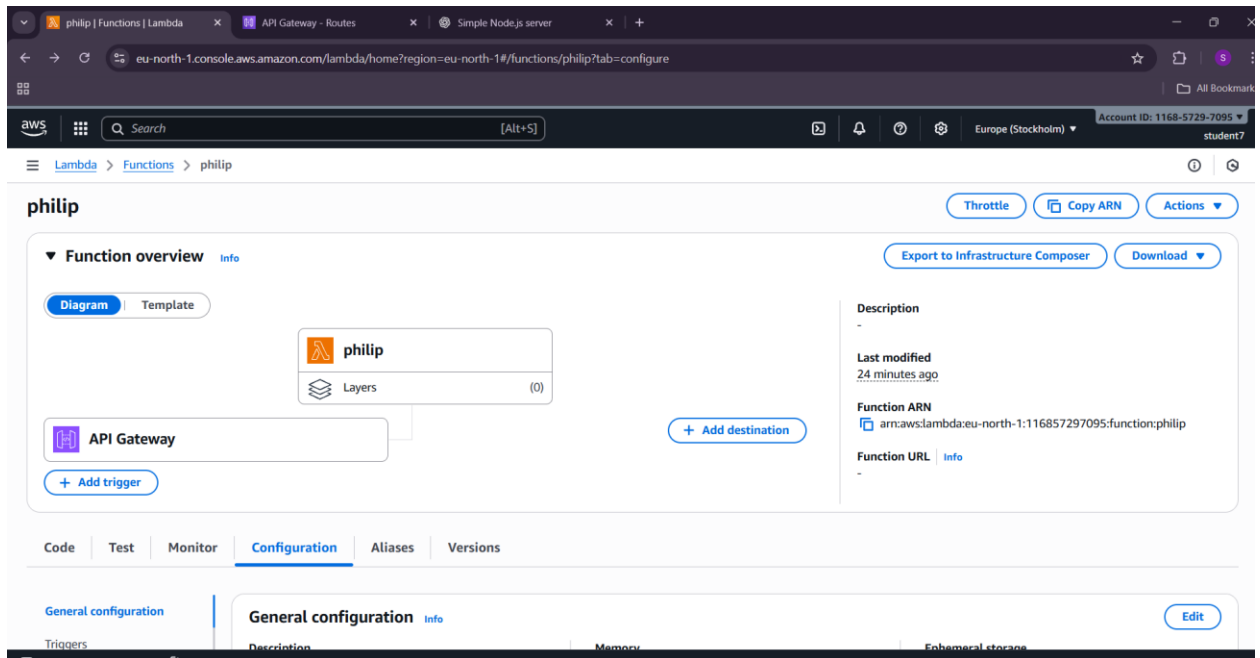
- PHILIP
 - express
 - node_modules
 - index.js
 - package-lock.json
 - package.json
 - DEPLOY
 - Deploy (Ctrl+Shift+U)
 - Test (Ctrl+Shift+I)
 - TEST EVENTS (NONE SELECTED)
 - Create new test event
 - ENVIRONMENT VARIABLES

index.js

```
1 const app = express();
2 app.use(express.json());
3
4 // Routes
5 app.get("/", (req, res) => {
6   res.json({ message: "Express running in single file on AWS Lambda!" });
7 });
8
9 app.get("/hello", (req, res) => {
10  res.json({ message: "Hi there!" });
11 });
12
13 app.post("/send", (req, res) => {
14  res.json({ received: req.body });
15 });
16
17 // Export Lambda handler
18 module.exports.handler = serverless(app);
```

3. Deployed API Gateway

- Deployed to a stage named dev.
- Obtained the invoke URL for testing.



4. Verified Functionality

- Opened browser and navigated to the API endpoint.
- Successfully retrieved the contents of the S3 object.

Conclusion:

AWS Lambda was successfully integrated with Amazon S3 via API Gateway. The Lambda function correctly accessed S3 objects, and the API Gateway endpoint provided external access to the data.

aws

Search

[Alt+S]

Europe (Stockholm)

Account ID: 1168-5729-7095

student7

Lambda > Functions > philip

Test event Info

CloudWatch Logs Live Tail

Save

Test

To invoke your function without saving an event, configure the JSON event, then choose Test.

Test event action

Create new event

Edit saved event

Invocation type

☒ Synchronous

Executes the Lambda function and blocks until receiving the function's response, with a maximum timeout of 15 minutes. Returns function output or error details directly to the calling application.

☐ Asynchronous

Enqueues the Lambda function for execution and returns immediately with a request ID. Function processes independently, with results optionally sent to a configured destination like SQS, SNS, or EventBridge.

Event name

testevent

Maximum of 25 characters consisting of letters, numbers, dots, hyphens and underscores.

Event sharing settings

☒ Private

This event is only available in the Lambda console and to the event creator. You can configure a total of 10. [Learn more](#)

☐ Shareable

This event is available to IAM users within the same account who have permissions to access and use shareable events. [Learn more](#)

Template - optional

Hello World

CloudShell

Feedback

Console Mobile App

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

aws

Search

[Alt+S]

Europe (Stockholm)

Account ID: 1168-5729-7095

student7

Lambda > Functions > philip

Successfully updated the function philip.

EXPLORER

PHILIP

express

node_modules

index.js

package-lock.json

package.json

DEPLOY

Current

Deploy (Ctrl+Shift+U)

Test (Ctrl+Shift+I)

TEST EVENTS [NONE SELECTED]

Create new test event

package-lock.json

package-lock.json > {} packages > {} -- > license

6 "packages": {

19 "node_modules/@isaacs/cliui": {

25 "dependencies": {

27 "strip-ansi": "npm:strip-ansi@^6.0.0",

28 "strip-ansi-cjs": "npm:strip-ansi@^6.0.1",

29 "wrap-ansi": "npm:wrap-ansi@^7.0.0",

30 "wrap-ansi-cjs": "npm:wrap-ansi@^7.0.0"

31 },

32 "engines": {

33 "node": ">=12"

34 },

35 },

36 "node_modules/@pkgjs/parseargs": {

37 "version": "0.11.0",

38 }

PROBLEMS

OUTPUT

CODE REFERENCE LOG

TERMINAL

Execution Results

ImportModuleError [Runtime.ImportModuleError]: Error: Cannot find module 'index'

Require stack:

- /var/runtime/index.mjs

at loadModule (file:///var/runtime/index.mjs:577:13)

CloudShell

Feedback

Console Mobile App

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

aws

Search

[Alt+S]

Europe (Stockholm)

Account ID: 1168-5729-7095

student7

Lambda > Functions > philip

Shareable

This event is available to IAM users within the same account who have permissions to access and use shareable events. [Learn more](#)

Template - optional

Hello World

Event JSON

Format JSON

```
1 {
2   "httpMethod": "GET",
3   "path": "/"
4 }
```

3:10 JSON

aws [Search] [Alt+S] Europe (Stockholm) Account ID: 1168-5729-7095 student7

API Gateway > APIs > Resources - philip-API (nmhdizdaaa)

Stages
Authorizers
Gateway responses
Models
Resource policy
Documentation
Dashboard
API settings

Usage plans
API keys
Client certificates
Settings

Developer portals **New**
Portals
Portal products

Successfully created method 'GET' in '/'. Redeploy your API for the update to take effect.

Resources

Create resource

/ GET
/philip ANY

API actions Deploy API

Update documentation Delete

ARN: arn:aws:execute-api:eu-north-1:116857297095:nmhdizdaaa:/GET/ Resource ID: 63dic5cgld

Client → Method request → Integration request → Lambda integration
← Method response ← Integration response Proxy integration ←

Method request Integration request Integration response Method response Test

API Gateway > APIs > Resources - philip-API (nmhdizdaaa) > Create method

Create method

Method details

Method type
Select a method type

Integration type

☒ **Lambda function**
Integrate your API with a Lambda function.

☐ **HTTP**
Integrate with an existing HTTP endpoint.

☐ **Mock**
Generate a response based on API Gateway mappings and transformations.

☐ **AWS service**
Integrate with an AWS Service.

☐ **VPC link**
Integrate with a resource that isn't accessible over the public internet.

☐ **Lambda proxy integration**
Send the request to your Lambda function as a structured event.

Resource transfer mode

